



College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID623001: Introductory Game Development
Level 6, Credits 15
Portfolio

Assessment Overview

In this **individual/group** assessment, you will develop **four** games. In addition, marks will be allocated for code quality and best practices, documentation and **Git** usage.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Design and build usable, attractive games using various introductory algorithms following an appropriate software development methodology.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Portfolio	100%	Monday 9th June at 7.59 AM	1

Conditions of Assessment

You will complete this assessment during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Monday 9th June at 7.59 AM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID623001: Introductory Game Development**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using an **AI generative tool**. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

Learning to use AI tool is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at <https://www.op.ac.nz/about-us/governance-and-management/policies>.

Submission

You **must** submit all application files via **GitHub Classroom**. Here is the URL to the repository you will use for your submission – <https://classroom.github.com/a/UvdYWSIH>. Late submissions will incur a **10% penalty per day**, rolling over at **8.00 AM**.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID623001: Introductory Game Development**.

Instructions

Functionality - Learning Outcome 1 (55%)

Pong - 10% (Individual)

- Build and publish the game to **itch.io**.
- Implement the fundamental mechanics of the game, including:
 - Main menu - The game should have a main menu with options to start the game, view the high scores and exit the game.
 - Paddle movement - The player should be able to move the paddle up and down using the **W** and **S** keys using the **Unity** input system.
 - Ball movement.
 - Collisions - The ball should bounce off the top and bottom walls and the paddles.
 - Score system - One point should be awarded to the player who scores a goal.
 - Score display - The scores of both players should be displayed on the screen.
 - High scores - The game should store and display the top **10** high scores.
 - Pause and resume - The player should be able to pause and resume the game using the **P** key.
 - Sound effects - The game should have sound effects for the ball bouncing off the walls and paddles, scoring a goal and the game ending.
 - Game over - The game should end when a player reaches a score of **10**.
 - End game menu - The game should have an end game menu with options to play again, view the high scores and exit the game.
- Tier 2 mechanics. Select **three** mechanics from the list below):
 - AI paddle - An AI paddle that moves up and down.
 - Difficulty levels - Easy, medium and hard difficulty levels that adjust the speed of the AI paddle.
 - Ball speed - The speed of the ball should increase after a certain number of goals.
 - Countdown timer - A countdown timer that starts the game after a certain number of seconds.
 - Customisation - Asset options for the paddles and ball.
 - Music - Background music that plays during the game.
- Tier 3 mechanics. Select **two** mechanics from the list below):
 - Multiple rounds - Best 3 out of 5 rounds to win the game.
 - Power-ups - Random drops that allow the player to increase the size of the paddle and slow down the ball.
 - Trail effects - Trail effects for the ball.
 - Sound settings - Options to adjust the volume of the sound effects and music.
- Three mechanics of your choice. These mechanics should be different from the ones listed above.

Breakout - 15% (Individual)

- Build and publish the game to **itch.io**.

- Implement the fundamental mechanics of the game, including:
 - Main menu - The game should have a main menu with options to start the game, view the high scores and exit the game.
 - Paddle movement - The player should be able to move the paddle left and right using the **A** and **D** keys using the **Unity** input system.
 - Ball movement.
 - Collisions - The ball should bounce off the walls, paddle and bricks.
 - Brick formation - The game should have a 10 x 4 grid of bricks.
 - Brick health - Each brick should have a health value that determines how many hits it can take before being destroyed.
 - Score system - One point should be awarded to the player who destroys a brick.
 - Score display - The score of the player should be displayed on the screen.
 - Live system - The player should have **3** lives.
 - Lives display - The lives of the player should be displayed on the screen.
 - High scores - The game should store and display the top **10** high scores.
 - Sound effects - The game should have sound effects for the ball bouncing off the walls, paddle and bricks, scoring a point and the game ending.
 - Pause and resume - The player should be able to pause and resume the game using the **P** key.
 - Game over - The game should end when the player destroys all the bricks or loses all their lives.
 - End game menu - The game should have an end game menu with options to play again, view the high scores and exit the game.
- Tier 2 mechanics. Select **three** mechanics from the list below:
 - Difficulty levels - Easy, medium and hard difficulty levels that adjust the speed of the ball and health of the bricks.
 - Ball speed - The speed of the ball should increase after a certain number of goals.
 - Countdown timer - A countdown timer that starts the game after a certain number of seconds.
 - Customisation - Asset options for the paddle and ball.
 - Music - Background music that plays during the game.
- Tier 3 mechanics. Select **two** mechanics from the list below:
 - Time limit - A time limit to complete the game.
 - Three levels - Three levels with different brick formations.
 - Moving bricks - Bricks that move horizontally.
 - Sound settings - Options to adjust the volume of the sound effects and music.
 - Animated effects - Animated destruction effects for the bricks.
 - Power-ups - Random drops that allow the player to shoot multiple balls, increase the size of the paddle and slow down the ball.
- Three mechanics of your choice. These mechanics should be different from the ones listed above.

Space Invaders - 15% (Individual)

- Build and publish the game to **itch.io**.
- Implement the fundamental mechanics of the game, including:
 -

- Main menu - The game should have a main menu with options to start the game, view the high scores and exit the game.
 - Ship movement - The player should be able to move the ship left and right using the **A** and **D** keys using the **Unity** input system.
 - Collisions - Collisions between the ship, bullets and enemies.
 - Enemy formation - The game should have a 10 x 4 grid of enemies.
 - Enemy movement - The enemies should move left and right and down when they reach the edge of the screen.
 - Shooting - The player and enemies should be able to shoot bullets.
 - Score system - One point should be awarded to the player who destroys an alien.
 - Score display - The score of the player should be displayed on the screen.
 - Live system - The player should have **3** lives.
 - Lives display - The lives of the player should be displayed on the screen.
 - Player name - The player should be able to enter their name before starting the game. If a name is not entered, get a random name from the **Random User Generator API**.
 - High scores - The game should store and display the top **10** high scores.
 - Sound effects - The game should have sound effects for the ship shooting bullets, the enemies shooting bullets, the enemies being destroyed, the ship being destroyed and the game ending.
 - Pause and resume - The player should be able to pause and resume the game using the **P** key.
 - Game over - The game should end when the player destroys all the enemies or loses all their lives.
 - End game menu - The game should have an end game menu with options to play again, view the high scores and exit the game.
- Tier 2 mechanics. Select **three** mechanics from the list below:
 - Difficulty levels - Easy, medium and hard difficulty levels that adjust the speed of the enemies and the number of enemies.
 - Bullet speed - The speed of the bullets should increase after a certain number of enemies are destroyed.
 - Customisation - Asset options for the ship, enemies, bullets and background.
 - Music - Background music that plays during the game.
 - UFO - A UFO that moves across the top of the screen and gives the player bonus points when destroyed.
 - Parallax scrolling - A parallax scrolling effect for the background.
 - Stat tracking - Statistics that track the player's accuracy and the number of enemies destroyed.
 - Accessibility settings - Dyslexia-friendly font and colour-blind mode.
 - Tier 3 mechanics. Select **two** mechanics from the list below:
 - Boss battles - Boss enemies that have unique attack patterns and abilities.
 - Bunkers - Bunkers that protect the player from alien bullets. The bunkers should have health values that decrease when hit by bullets.
 - Animated effects - Animated destruction effects for the enemies and ship.
 - Story mode - A simple narrative that explains the player's mission.
 - Sound settings - Options to adjust the volume of the sound effects and music.
 - Enemy formations - Different alien formations that move in unique patterns.
 - Achievement system - Achievements that reward the player for completing specific tasks.
 - Three mechanics of your choice. These mechanics should be different from the ones listed above.

RogueLike - 15% (Group)

- Build and publish the game to **itch.io**.
- Implement the fundamental mechanics of the game, including:
 - Main menu - The game should have a main menu with options to start the game, view the high scores and exit the game.
 - Player movement - The player should be able to move up, down, left and right using the **W**, **S**, **A** and **D** keys using the **Unity** input system.
 - Enemy movement - The enemies should move towards the player when they are in range.
 - Combat - The player and enemies should be able to attack each other.
 - Health system - The player and enemies should have health values that decrease when attacked.
 - Score system - One point should be awarded to the player who defeats an enemy.
 - Score display - The score of the player should be displayed on the screen.
 - Live system - The player should have **3** lives.
 - Lives display - The lives of the player should be displayed on the screen.
 - Inventory system - The player should be able to pick up items that increase their health and damage.
 - High scores - The game should store and display the top **10** high scores.
 - Sound effects - The game should have sound effects for the player attacking, the enemies attacking, the player being attacked and the game ending.
 - Pause and resume - The player should be able to pause and resume the game using the **P** key.
 - Game over - The game should end when the player defeats all the enemies or loses all their lives.
 - End game menu - The game should have an end game menu with options to play again, view the high scores and exit the game.
- Tier 2 mechanics. Select **three** mechanics from the list below:
 - Difficulty levels - Easy, medium and hard difficulty levels that adjust the health and damage of the enemies.
 - Item variety - Different items that provide unique effects to the player.
 - Customisation - Asset options for the player, enemies and items.
 - Music - Background music that plays during the game.
 - Random generation - Randomly generated levels that change each playthrough.
 - Events - Random events that occur during the game.
 - Multiple levels - Multiple levels with different enemy types and layouts.
- Tier 3 mechanics. Select **two** mechanics from the list below:
 - Boss battles - Boss enemies that have unique attack patterns and abilities.
 - Animated effects - Animated effects for the player attacking, the enemies attacking and the game ending.
 - Sound settings - Options to adjust the volume of the sound effects and music.
 - Fog of war - A fog of war effect that hides parts of the map until the player explores them.
 - Minimap - A minimap that shows the layout of the level and the player's position.
 - Teleportation - Teleportation pads that allow the player to move between different parts of the level.
- Four mechanics of your choice. These mechanics should be different from the ones listed above.

Code Quality and Best Practices - Learning Outcome 1 (40%)

- **Project Structure** - The codebase should be well-organised with a clear and logical structure that separates concerns and promotes reusability and maintainability.
- **Naming Conventions** - File, class, method, variable, constant and property names should be clear, descriptive and consistent across the codebase.
- **Documentation and Comments** - The codebase should be well-documented using the **XML** documentation format with comments that explain complex logic and clarify the purpose of classes, methods or complex sections.
- **Code Formatting** - Consistent indentation and spacing should be used across the codebase. It includes ensuring proper indentation for code blocks, following a standard format for braces and adding blank lines between sections.
- **Dead Code** - The codebase should not contain unused or redundant files, classes, methods, variables, constants or properties. Keeping unnecessary codebase around can lead to confusion, clutter and potential bugs down the line.
- **Performance and Scalability** - The codebase should be optimised for performance, using efficient algorithms and data structures to minimise bottlenecks.

Documentation and Git Usage - Learning Outcome 1 (5%)

- **GitHub** issues to help you organise and prioritise your development work. The course lecturer needs to see consistent use of **GitHub** issues for the duration of the assessment.
- Provide URLs to the games on **itch.io** in your repository **README.md** file
- Use of **Markdown**, i.e., headings, bold text, code blocks, etc.
- Correct spelling and grammar.
- A **.gitignore** file containing the ignored files in this resource - <https://raw.githubusercontent.com/github/gitignore/main/Unity.gitignore>.
- Your **Git commit messages** should:
 - Reflect the context of each functional requirement change.
 - Be formatted using an appropriate naming convention style.

Additional Information

- **Do not** rewrite your **Git** history. It is important that the course lecturer can see how you worked on your assessment over time.
- You need to show the course lecturer the initial **GitHub** issues before you start your development work. Following this, you need to show the course lecturer your **GitHub** issues at the end of each week.